SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	D. Sri Anjaneya	**Reg. No.:**	192425219
Section / Batch:	SLOT-D / 2024	**Date:**	15-07-2026

Code of Conduct

*I **D. Sri Anjaneya (192425219)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: *D. Sri Anjaneya*

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:
Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
Password
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)
Mobile Number
Must contain exactly 10 digits.
First digit should be between 6–9.
Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
    - States
    - Input alphabet
    - Transition table
    - Initial state
    - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO1 – Assessment Tool 3: Coding Challenge

Answers & Source Code

Name: D. Sri Anjaneya

Section 1: Intelligent Email and Password Validator using Regular Expressions

Python Solution: -

```
import re

def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    return re.match(pattern, email) is not None

def validate_password(password):
    if len(password) < 8:
        return False
    if not re.search(r'[A-Z]', password):
        return False
    if not re.search(r'[a-z]', password):
        return False
    if not re.search(r'[0-9]', password):
        return False
    if not re.search(r'[@#$%&!]', password):
        return False
    return True

def validate_mobile(number):
    pattern = r'^[6-9][0-9]{9}$'
    return re.match(pattern, number) is not None

def main():
    test_emails = ["john.doe@gmail.com", "1john@gmail.com", "john@123.com"]
    test_passwords = ["Pass@123", "password", "PASSWORD123"]
    test_mobiles = ["9876543210", "1234567890", "98765432"]

    print("--- Email Validation ---")
    for e in test_emails:
        result = "Valid Email" if validate_email(e) else "Invalid Email"
        print(f"{e}: {result}")

    print("\n--- Password Validation ---")
    for p in test_passwords:
        result = "Strong Password" if validate_password(p) else "Weak Password"
        print(f"{p}: {result}")

    print("\n--- Mobile Number Validation ---")
    for m in test_mobiles:
        result = "Valid Mobile Number" if validate_mobile(m) else "Invalid Mobile Number"
        print(f"{m}: {result}")

if __name__ == "__main__":
    main()
```

Sample Output: -

```
--- Email Validation ---
john.doe@gmail.com: Valid Email
1john@gmail.com: Invalid Email
john@123.com: Invalid Email

--- Password Validation ---
Pass@123: Strong Password
password: Weak Password
PASSWORD123: Weak Password

--- Mobile Number Validation ---
9876543210: Valid Mobile Number
1234567890: Invalid Mobile Number
98765432: Invalid Mobile Number
```

Section 2: Design a Finite State Automata (FSA) Simulator

Python Solution: -

```
class DFA:
    def __init__(self, states, alphabet, transitions, start_state, final_states):
        self.states = states
        self.alphabet = alphabet
        self.transitions = transitions
        self.start_state = start_state
        self.final_states = final_states

    def run(self, input_string):
        current_state = self.start_state
        path = [current_state]

        for symbol in input_string:
            if symbol not in self.alphabet:
                return path, False, f"Symbol '{symbol}' not in alphabet"
            key = (current_state, symbol)
            if key not in self.transitions:
                return path, False, "No valid transition"
            current_state = self.transitions[key]
            path.append(current_state)

        accepted = current_state in self.final_states
        return path, accepted, None

def main():
    # DFA for strings ending with "ab"
    states = {"q0", "q1", "q2"}
    alphabet = {"a", "b"}
    transitions = {
        ("q0", "a"): "q1",
        ("q0", "b"): "q0",
        ("q1", "a"): "q1",
        ("q1", "b"): "q2",
        ("q2", "a"): "q1",
        ("q2", "b"): "q0",
    }
    start_state = "q0"
    final_states = {"q2"}

    dfa = DFA(states, alphabet, transitions, start_state, final_states)

    test_strings = ["abaab", "aab", "ba", "abab"]

    for s in test_strings:
        path, accepted, error = dfa.run(s)
        print(f"Input: {s}")
        if error:
            print(f"Error: {error}")
        else:
            print("Transition Path:")
            print(" -> ".join(path))
            print("Accepted" if accepted else "Rejected")
        print()

if __name__ == "__main__":
    main()
```

Sample Output: -

```
Input: abaab
Transition Path:
q0 -> q1 -> q0 -> q1 -> q1 -> q2
Accepted

Input: aab
Transition Path:
q0 -> q1 -> q1 -> q2
Accepted

Input: ba
Transition Path:
q0 -> q0 -> q1
Rejected

Input: abab
Transition Path:
q0 -> q1 -> q2 -> q1 -> q2
Accepted
```

Section 3: Smart Pattern Matching Engine using Regular Expressions

Python Solution: -

```
import re

def search_word(text, word):
    pattern = r'\b' + re.escape(word) + r'\b'
    return re.findall(pattern, text, re.IGNORECASE)

def search_prefix(text, prefix):
    pattern = r'\b' + re.escape(prefix) + r'\w*'
    return re.findall(pattern, text, re.IGNORECASE)

def search_suffix(text, suffix):
    pattern = r'\w*' + re.escape(suffix) + r'\b'
    return re.findall(pattern, text, re.IGNORECASE)

def search_date(text):
    pattern = r'\b\d{2}/\d{2}/\d{4}\b'
    return re.findall(pattern, text)

def search_phone(text):
    pattern = r'\b[6-9]\d{9}\b'
    return re.findall(pattern, text)

def search_hashtag(text):
    pattern = r'#\w+'
    return re.findall(pattern, text)

def search_mention(text):
    pattern = r'@\w+'
    return re.findall(pattern, text)

def main():
    sample_text = """Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing"""

    menu = """
User Menu
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
"""
    print(menu)
    choice = "1"   # example run; replace with input() interactively

    if choice == "1":
        print("Dates found:", search_date(sample_text))
    elif choice == "2":
        print("Phone numbers found:", search_phone(sample_text))
    elif choice == "3":
        print("Hashtags found:", search_hashtag(sample_text))
```

```
    elif choice == "4":
        print("Mentions found:", search_mention(sample_text))
    elif choice == "5":
        prefix = "nat"
        print(f"Words with prefix '{prefix}':", search_prefix(sample_text, prefix))
    elif choice == "6":
        suffix = "ing"
        print(f"Words with suffix '{suffix}':", search_suffix(sample_text, suffix))

if __name__ == "__main__":
    main()
```

Sample Output: -

```
User Menu
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix

Dates found: ['12/09/2026']

(Running choice 2) Phone numbers found: ['9876543210']
(Running choice 3) Hashtags found: ['#NLP']
(Running choice 4) Mentions found: ['@OpenAI']
(Running choice 5, prefix 'nat') Words with prefix 'nat': ['natural']
(Running choice 6, suffix 'ing') Words with suffix 'ing': ['processing']
```